TTG across 15 applications

Current runtime f9a6d37b2 · GIL-off Python 3.14.6 · asynchronous thread prefetch

- 15 fixed applications; 120 fresh workflows: 30 excluded validation warmups + 90 measured.
- All correctness and GIL-off checks pass. HTTP median improves in 1/15 selected workflows.
- Compiler support, observed prefetch coverage and measured speedup are evaluated separately.
- Every slower result is retained; targeted diagnostics explain observed costs.

The question and the fixed experiment

Select by source structure before looking at speed; keep the complete business workflow

- Same runtime and source in off/thread; one worker, limit 2048, seed 42, fresh database per trial.

- JDrive: 364 folders + 121 files. LittleX: 50 profiles + 1,000 tweets. Other 13: original seeded domain scenarios.

- One uniform ANALYZE before primary/reset. Setup, steady checks and restart are retained but excluded.

- Three measured repetitions per cell, rotated off/thread order. Report medians and ranges, not population estimates.

From program dependencies to prefetched data

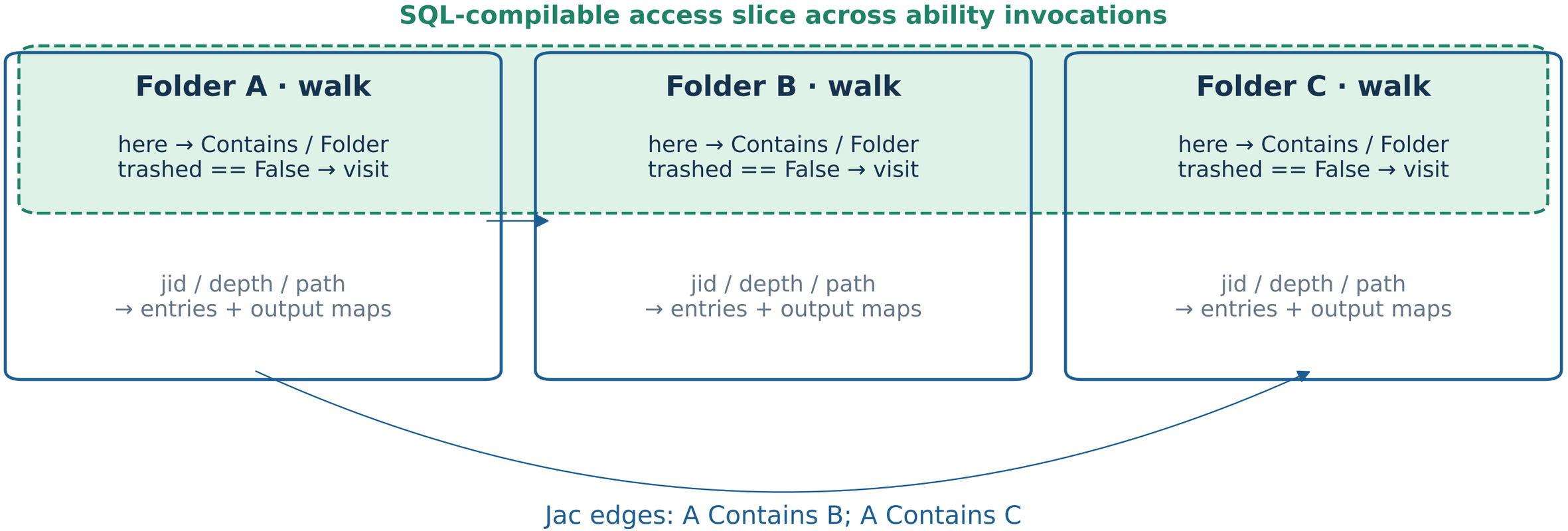
Native abilities still execute; compiler/runtime prediction changes fetching



Output-only bookkeeping can be excluded from the access slice only with an effect contract. Unknown effects, access-dependent shared state and cancellation remain dependencies.

Jac graph, with dependencies inside each node

Illustrative Folder A → Folder B / Folder C; same native walk ability at every Folder



The green slice predicts accesses. Grey output dependencies remain in native execution; they do not control the visit predicate in this source.

What “supported” means

Four separate observations; one cannot stand in for another

Level	Evidence required
Correctness	Native outputs, mutations, negative cases, restart and physical state agree
Predictable access	Typed path / supported predicate; output effects cannot alter later access
Active optimization	Actual region/template query, cache reuse and background SQL records
Performance benefit	All-role SQL + DB intervals + complete HTTP workflow improve

Cost model: eligibility is only the first decision

This experiment uses AlwaysPrefetch to expose costs; it does not claim learned admission gains

- An eligible region predicts which accesses can be fetched. It does not guarantee that fetching them now is profitable.

- A useful admission estimate compares saved foreground waits with added path SQL, payload work, runtime planning and executor setup.

- Cache hits, region size, query shape and expected unused data must enter the estimate; actual concurrency should be measured.

- The present off/Always-thread comparison provides evidence for admission thresholds. Prepared-statement reuse and adaptive admission remain follow-up work.

Why these applications are useful tests

Structural candidates plus explicit partial-support controls

Family	Selected applications	What it exercises
Recursive tree	JDrive	Boolean-filtered tree, output maps, ordered cache
Cross-ability graph	LittleX	Profile/Post/Follow region; payload and mutation costs
Typed chains / branches	IssueDesk, StockRoom, ClinicQueue, BudgetLedger	Multiple visits and typed dispatch; helper boundaries
Typed branches + nested reads	LibraryLoans, HelpQueue, DeliveryRoute, VolunteerShift	Root/branch prefixes versus terminal helper/loop reads
Short prefix / output-heavy	ApprovalDesk, EventSeats, KnowledgeMap, CoursePath, IncidentRoom	Low opportunity and partial coverage retained

Whole workflow results — 1 to 5

Medians of n=3; milliseconds; SQL includes foreground, background and controls

Application	HTTP off → thread	Thread / off	DB off → thread	SQL off → thread
JDrive	398.90 → 217.76	0.55×	209.92 → 41.07	463 → 105
LittleX	215.72 → 275.59	1.28×	50.11 → 60.40	153 → 143
IssueDesk	278.59 → 370.20	1.33×	145.93 → 180.51	557 → 565
StockRoom	266.49 → 388.27	1.46×	134.97 → 161.90	497 → 493
ClinicQueue	212.75 → 229.43	1.08×	108.60 → 110.82	408 → 411

Whole workflow results — 6 to 10

Medians of n=3; milliseconds; SQL includes foreground, background and controls

Application	HTTP off → thread	Thread / off	DB off → thread	SQL off → thread
BudgetLedger	119.69 → 147.65	1.23×	57.04 → 64.36	234 → 236
KnowledgeMap	145.67 → 183.90	1.26×	72.24 → 88.16	298 → 301
CoursePath	291.09 → 328.09	1.13×	154.17 → 175.32	521 → 519
ApprovalDesk	264.75 → 404.40	1.53×	134.81 → 158.94	539 → 545
LibraryLoans	294.88 → 340.67	1.16×	152.60 → 164.82	531 → 533

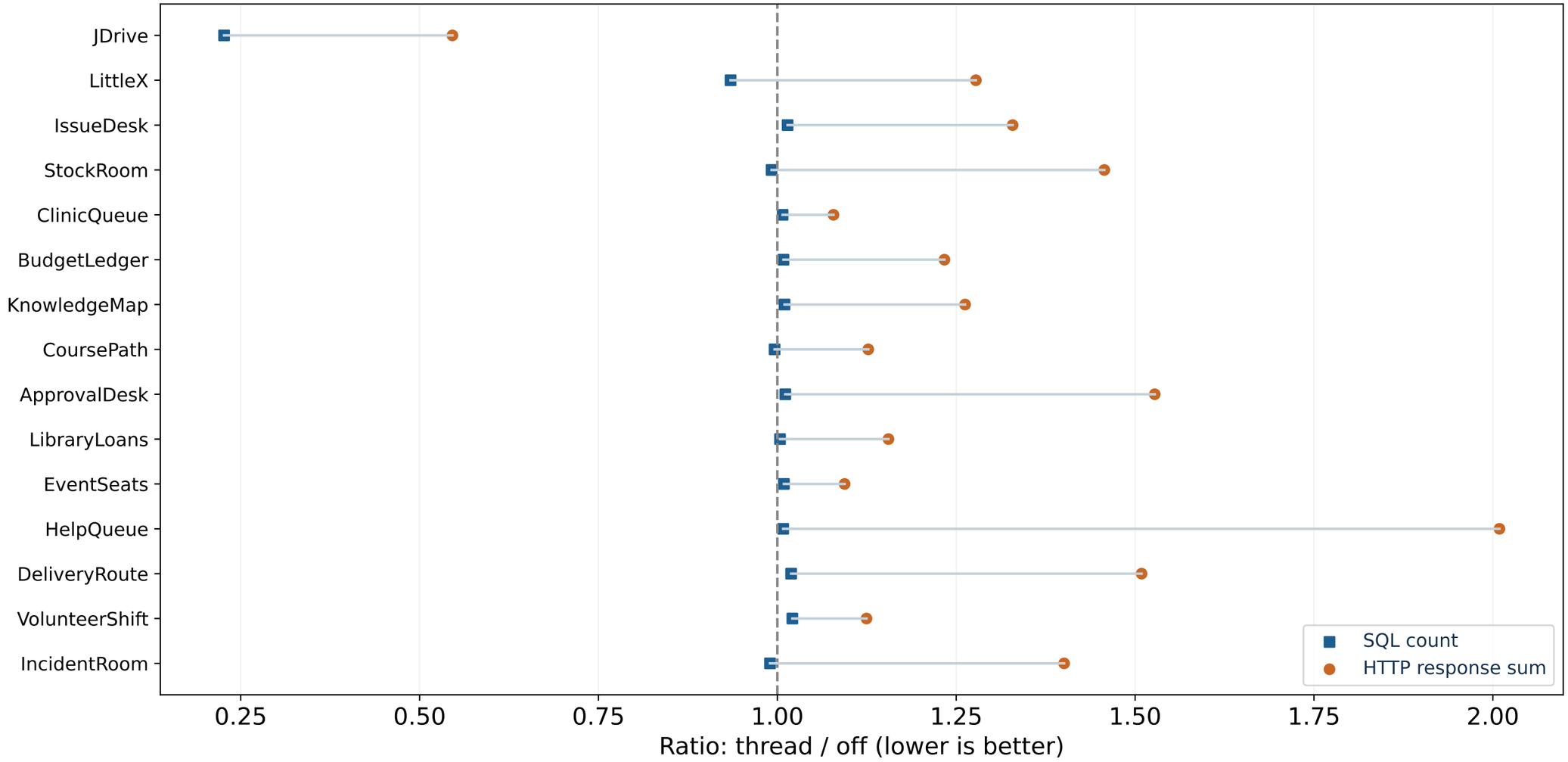
Whole workflow results — 11 to 15

Medians of n=3; milliseconds; SQL includes foreground, background and controls

Application	HTTP off → thread	Thread / off	DB off → thread	SQL off → thread
EventSeats	220.03 → 240.72	1.09×	110.51 → 114.23	434 → 438
HelpQueue	321.17 → 645.30	2.01×	170.57 → 193.29	609 → 614
DeliveryRoute	215.34 → 324.96	1.51×	103.19 → 135.18	415 → 423
VolunteerShift	161.67 → 181.81	1.12×	85.97 → 87.28	286 → 292
IncidentRoom	141.89 → 198.77	1.40×	68.58 → 91.21	293 → 290

SQL reduction is not a speedup guarantee

Each point is one application; medians, n=3 per mode



HTTP variability — 1

Observed minimum–maximum of three primary workflows; milliseconds

Application	Off min–max	Thread min–max
JDrive	395.77–507.92	213.48–227.77
LittleX	215.12–216.21	275.18–276.53
IssueDesk	277.31–287.45	319.85–391.33
StockRoom	263.62–270.56	381.95–395.24
ClinicQueue	208.74–220.53	227.97–251.83
BudgetLedger	114.39–155.81	143.41–152.83
KnowledgeMap	142.81–148.98	173.36–200.66
CoursePath	289.91–293.36	327.95–345.82

HTTP variability — 9

Observed minimum–maximum of three primary workflows; milliseconds

Application	Off min–max	Thread min–max
ApprovalDesk	260.89–286.83	382.60–433.49
LibraryLoans	284.20–390.35	338.54–346.45
EventSeats	215.36–225.06	232.24–298.38
HelpQueue	310.24–692.94	617.13–649.67
DeliveryRoute	205.68–264.44	263.60–345.98
VolunteerShift	160.26–177.09	175.30–192.32
IncidentRoom	141.86–202.64	186.14–230.79

Reader-only results — 1

Sum of primary registered-reader calls within each workflow; no steady-read samples reused

Application	Reader calls	Reader SQL off → thread	Reader HTTP ms off → thread
JDrive	3	403 → 45	365.61 → 183.31
LittleX	4	68 → 56	142.20 → 198.79
IssueDesk	5	116 → 124	65.57 → 94.43
StockRoom	8	206 → 200	120.33 → 181.84
ClinicQueue	4	104 → 107	60.22 → 76.07
BudgetLedger	4	75 → 77	40.20 → 65.31
KnowledgeMap	4	99 → 102	54.91 → 67.77
CoursePath	4	97 → 95	56.92 → 71.04

Reader-only results — 9

Sum of primary registered-reader calls within each workflow; no steady-read samples reused

Application	Reader calls	Reader SQL off → thread	Reader HTTP ms off → thread
ApprovalDesk	4	82 → 85	42.90 → 54.59
LibraryLoans	5	134 → 136	84.71 → 116.64
EventSeats	5	120 → 124	67.63 → 80.83
HelpQueue	6	150 → 150	84.08 → 126.69
DeliveryRoute	5	107 → 115	57.62 → 117.90
VolunteerShift	4	91 → 97	52.15 → 74.47
IncidentRoom	5	97 → 94	51.00 → 71.95

Observed optimization coverage — 1

Actual primary counters; cumulative demand counts are not distinct graph-node counts

Application	Static queries	Templates	Cache reuses	Submitted / promoted
JDrive	3	0	258	258 / 258
LittleX	6	0	4	287 / 287
IssueDesk	5	4	0	14 / 14
StockRoom	8	9	0	44 / 44
ClinicQueue	4	0	0	3 / 3
BudgetLedger	4	3	0	10 / 10
KnowledgeMap	4	0	0	9 / 9
CoursePath	4	0	0	5 / 5

Observed optimization coverage — 9

Actual primary counters; cumulative demand counts are not distinct graph-node counts

Application	Static queries	Templates	Cache reuses	Submitted / promoted
ApprovalDesk	4	4	0	9 / 9
LibraryLoans	5	4	0	19 / 19
EventSeats	5	0	0	4 / 4
HelpQueue	9	8	0	30 / 30
DeliveryRoute	5	4	0	16 / 16
VolunteerShift	4	3	0	9 / 9
IncidentRoom	5	0	0	14 / 14

JDrive: a strong structural match

One homogeneous typed directory traversal, boolean predicate and private output state

- visit Contains → Folder(trashed == False) determines access; jid/depth/path bookkeeping constructs output.

- The certificate permits the supported private output grammar while preserving every native report and visit.

- Recursive discovery joins worktable → edge → target; ordered cache publication checks completeness and graph validity.

- This run: SQL 463 → 105; HTTP 398.90 → 217.76 ms.

HelpQueue: where the extra time appears

Primary evidence identifies intervals and work; causal claims need separate diagnostics

- HTTP 321.17 → 645.30 ms; DB 170.57 → 193.29 ms.

- Data SQL 413 → 402; controls 196 → 212.

- Largest per-walker summed HTTP increase: EvaluateSLA, 29.54 → 298.67 ms; its SQL 63 → 68 and DB 15.21 → 23.27 ms.

- These are attributed protocol/response intervals; they do not identify CPU or synchronization time by subtraction. Separate matched diagnostics follow.

ApprovalDesk: where the extra time appears

Primary evidence identifies intervals and work; causal claims need separate diagnostics

- HTTP 264.75 → 404.40 ms; DB 134.81 → 158.94 ms.

- Data SQL 343 → 343; controls 196 → 202.

- Largest per-walker summed HTTP increase: SubmitExpense, 57.03 → 151.41 ms; its SQL 109 → 112 and DB 29.85 → 35.90 ms.

- These are attributed protocol/response intervals; they do not identify CPU or synchronization time by subtraction. Separate matched diagnostics follow.

DeliveryRoute: where the extra time appears

Primary evidence identifies intervals and work; causal claims need separate diagnostics

- HTTP 215.34 → 324.96 ms; DB 103.19 → 135.18 ms.

- Data SQL 268 → 268; controls 147 → 155.

- Largest per-walker summed HTTP increase: RouteBoard, 57.62 → 117.90 ms; its SQL 107 → 115 and DB 28.89 → 41.08 ms.

- These are attributed protocol/response intervals; they do not identify CPU or synchronization time by subtraction. Separate matched diagnostics follow.

HelpQueue: expensive prediction, little saved work

Matched diagnostic: all three EvaluateSLA requests; direct same-thread CPU timers

- Native-worker CPU sum: off 8.72 ms; thread 192.06 ms. Full original workflow and state checks pass.

- plan_continuation alone: 19 calls, 173.00 ms exclusive CPU. Nested timers are partitioned; inclusive recursive totals are not added.

- Primary EvaluateSLA stops at the ticket conditional/graph-read barrier. It still invokes continuation prediction and frontier resolution; only a short prefix is prefetched.

- This directly locates substantial extra runtime work. It does not apportion the entire primary HTTP difference or measure PostgreSQL CPU.

ApprovalDesk: the exit phase limits the region

Matched diagnostic: all five SubmitExpense requests, including native rejection cases

- Native-worker CPU sum: off 17.01 ms; thread 112.38 ms. plan_continuation: 29 calls / 77.45 ms exclusive CPU.

- LineItem values accumulate into self.total. Expense exit then chooses approval roles, traverses approvers and creates ApprovalStep nodes.

- Actual primary fallback says: Walker exit effects are outside the static region kernel. The line-item prefix can be fetched; the later routing/write phase remains procedural.

- Compiler eligibility for one prefix is insufficient for profitability. Avoid repeated planning at known unsupported boundaries; price planner CPU in admission.

LittleX: construction and execution still cost work

First load_feed diagnostic; declared inputs and full native program unchanged

- Native-worker CPU: off 19.85 ms; thread 29.69 ms. 65 observed event hooks remain.

- Named TTG runtime work totals 12.23 ms exclusive CPU. Static region resolution is attempted 62 times, including later cache/terminal checks.

- Primary workflow SQL falls 153 → 143, yet DB median rises 50.11 → 60.40 ms and HTTP 215.72 → 275.59 ms.

- The matched SQL plans on the next page show why fewer SELECTs need not mean less server work. These diagnostic timings are not primary n=3 measurements.

LittleX SQL: one complex query versus three simple paths

Same logical state, actual Bind, uniform ANALYZE; census-warmed matched EXPLAIN

- First plans: off planning 1.188 / execution 0.469 ms total; thread 2.696 / 1.431 ms.

- Immediate repeat: off 1.121 / 0.385 ms; thread 1.897 / 1.147 ms. Root shared hits 329 → 513; all physical reads zero.

- Compiled plan adds six WindowAgg nodes, ordering/CTE/aggregate machinery and an anchors Seq Scan (1,104 rows, 1 loop). Ordinary paths use endpoint primary-key lookups.

- The compiled result includes path/cache metadata and markers: 64 records versus 1 + 20 + 40 native path rows. This is not a claim of 64 distinct fetched nodes.

What these diagnostics establish

Separate mechanisms; no single unexplained “thread overhead” label

- Runtime: the direct continuation timers establish a large CPU cost in the two slowest procedural workflows, even with small graphs and compiler-emitted templates.

- Database: matched plans establish extra planning/execution and buffer work for the LittleX combined query in this controlled warm regime.

- Scheduling: all 90 primary samples have zero observed same-request foreground/background SQL overlap. Actual GIL-off threads do not guarantee hidden DB latency.

- Future experiments: skip known-no-benefit regions before runtime interpretation, reuse safe analysis facts, specialize SQL metadata/ACL work, then test cost-based admission. No proposed speedup is counted here.

Threads do not automatically hide latency

Actual client/backend/thread evidence, not an executor label

- No foreground/background SQL protocol overlap in any of the90 measured workflows.

- All actual SQL-boundary records assert Py_GIL_DISABLED == 1 and GIL disabled; background calls use real distinct threads and PostgreSQL backends.

- HTTP sum ends after response reading; explicit drain is separate. DB sum and union are protocol wall times, not server CPU.

- HTTP minus DB is not a CPU estimate. Native-worker CPU boundary timers and server plans are separate diagnostic samples.

Why the remaining applications are not declared unsupported

15 unmeasured applications remain in the 30-app registry

LinkedList, Jacord, RecipePantry, TripBoard, HiringPipeline, SalesPipeline, GrantReview, LabSamples, AssetCare, UtilityMeter, PollAssembly, BuildRelease, ReturnCounter, GardenPlan, Tournament

- Typed root visits may be supported even when nested comprehensions or helper reads are not fused.

- Seen/visited counters, sibling state that changes later access, dynamic calls, custom descriptors and writes require explicit contracts.

- The prior 30-app sibling audit is conditional source analysis. It is not proof that the current compiler optimizes every reader.

Sibling dependency remains a real boundary

Output dependence and future-access dependence are different

- Safe candidate: append this node’s fields to a result list; the list never affects which nodes are visited next.

- Access-dependent: increment a shared visited count, then disengage at a global node budget; sibling order changes future traversal.

- A seen set used before visit or return can affect later access on diamonds/cycles. It cannot be treated as mere output bookkeeping.

- Current backward slicing includes possible writers across abilities and retains unknown effects. Native semantics are not simplified for these tests.

Next steps justified by the measurements

Proposals, not unmeasured speedup claims

- Make admission account for native planning, executor and extra control costs; decline tiny regions when saved waits are insufficient.

- Optimize repeatedly expensive SQL shapes using matched plans, not query count alone; preserve type/ACL/order checks.

- Extend proven helper and loop summaries where output reads dominate; require access-dependency and effect proof.

- Evaluate thread/connection reuse and prepared statements as separate versions; preserve session/actor/transaction isolation.

Reproduce and interpret the evidence

All 15 applications and all negative performance outcomes are retained

- 120 fresh workflows; 30 excluded validation warmups; 90 measured; 30 cells with n=3.

- Each accepted database is independently replayed and physically checked before owned active-zero cleanup. Raw files, actual GIL records, source hashes and shutdown evidence are retained.

- Whole-workflow and reader results include all foreground/background SQL and control statements. Synthetic seeded domain scenarios are small, not production scale.

- Raw audit, exact vectors, source support map, diagnostic plans/profiles and editable renderer accompany this PDF.